

bazinga

One malloc to rule them all

Summary:
In this exercise, you will allocate a two-dimensional grid with one single malloc, and display a word right in the middle of it.

Version: 1.0

Contents

I Introduction **2**

II Common Instructions **3**

III AI Instructions **4**

IV Mandatory part **5**

IV.1 Technical considerations 5

IV.2 The header 6

IV.3 One single line 6

IV.4 Expected output 6

IV.5 Examples 8

V Submission and peer-evaluation **9**

Chapter I

Introduction

Pasadena, California. Dr. Sheldon Cooper has decided that his favourite word deserves a proper banner: centered, perfectly framed, and available in any size.

Leonard offered to help. His plan: one `malloc` for each line of the banner, all stored in an array of pointers. Sheldon was horrified:

"Multiple allocations? Scattered all over the heap? In MY apartment? Memory is meant to be contiguous, Leonard. One request, one block, one line. Anything else is chaos."

So here we are: one single allocation, one single line of memory, and a two-dimensional banner hidden inside it.

Bazinga!



Fun fact: this is how most images are stored in memory. A 1920 x 1080 screen is nothing more than a single line of 2,073,600 pixels, cut into rows when it is displayed.

Chapter II

Common Instructions

- This exercise must be written in C.
- Your code must be written in accordance with the Norm. It will be checked with `norminette`, and you will receive a 0 if there is a Norm error.
- Your program must compile with `cc` and the flags `-Wall`, `-Wextra` and `-Werror`. If it does not compile, you will get 0.
- Your program must not quit unexpectedly (segmentation fault, bus error, double free, etc.). If this happens, your exercise will be considered non-functional and will receive a 0.
- All heap-allocated memory must be properly freed before your program ends. Memory leaks will not be tolerated.
- Declaring global variables is strictly forbidden.
- You may only use the external functions listed in this subject. Using a forbidden function is considered cheating. Cheaters get -42, and this grade is non-negotiable.
- All your output must be done with `write`: `printf`, `fprintf`, `putchar` and the like are forbidden in every exercise.
- No Makefile is required: your program will be compiled with the command given in Chapter IV.
- We encourage you to test your program with many different inputs, including edge cases (0, tiny grids, even and odd sizes, letters...). Your evaluators will do it, for sure.
- Submit your work to the assigned Git repository. Only the work in the Git repository will be graded.

Chapter III

AI Instructions

Context

This exercise is designed to help you consolidate the fundamental building blocks of your 42 training. To properly anchor these skills, you need to struggle with them yourself: true learning requires genuine intellectual effort, through challenge, repetition and peer-learning exchanges.

Main message

- ☛ Build strong foundations without shortcuts.
- ☛ The learning journey is more important than the result.
- ☛ An exercise solved by an AI is an exercise you did not solve.

Learner rules

- You should apply reasoning to your exercise, especially before turning to AI.
- You should not ask for direct answers to the AI.
- You must be able to explain every line you submit.

✓ Good practice:

> I draw a 4 x 3 grid on a sheet of paper and write the number of each cell in it, from 0 to 11. I look at the cells of the same column and notice how to go from a line and a column to a number. I get it.

✗ Bad practice:

> I ask an AI to center a word in a grid. It allocates one line at a time, which is forbidden, and I can't explain its formula anyway. I fail. During the exam (no AI) I'm stuck again. I fail.

Chapter IV

Mandatory part

Turn-in directory	`ex03/`
Files to Submit	`bazinga.c, bazinga.h`
Compilation	`cc -Wall -Wextra -Werror bazinga.c`
Arguments	`<width> <height>`
External Function	`malloc, free, write`
Description	`Builds a grid with one single malloc and displays BAZINGA in its middle.`

IV.1 Technical considerations

✓ **Allowed:**

- The external functions listed above.
- Helper functions (your own `ft_strlen`, `ft_atoi`...), within the limits of the Norm.

✗ **Forbidden:**

- More than **one** call to `malloc` in your whole program. `calloc`, `realloc` and any other allocation function are forbidden too.
- Any other external function (`printf`, `putchar`, `strlen`, `atoi`...).
- Arrays (`char grid[10][10]`, variable-length arrays...) and arrays of pointers (`char **`): your grid must be one single line of memory.
- Displaying anything before the grid is completely built in memory.
- Global variables.



`One malloc. Not two. Not one per line. One.`

IV.2 The header

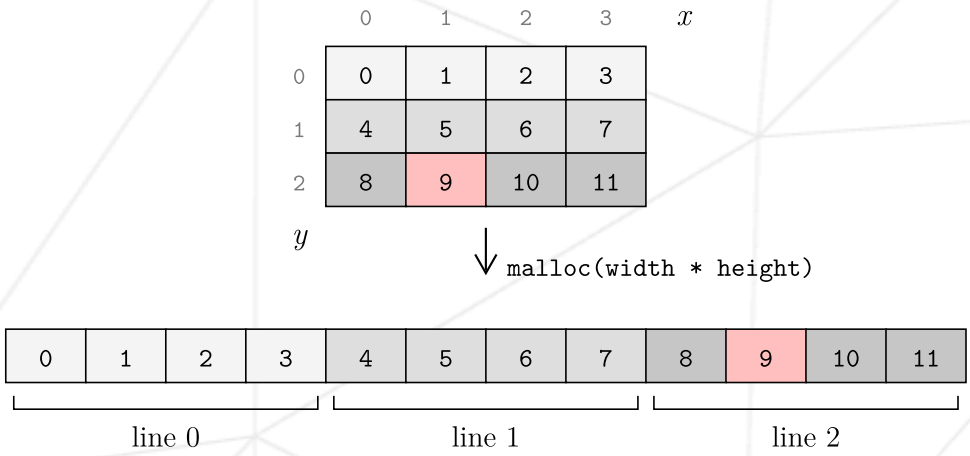
Your `bazinga.h` file must be protected against double inclusion, include the headers you need, and define the following two macros:

```
# define BAZINGA    "BAZINGA"
# define FILLER     '.'
```

Your program must only use these macros, never the word or the character themselves. It must work with any other value: any non-empty word for `BAZINGA`, any printable character for `FILLER`.

IV.3 One single line

With one single `malloc`, your grid is nothing more than a line of `width * height` characters. It only looks like a grid because you cut it into lines when you display it:



*Cell (x = 1, y = 2) of a 4 x 3 grid is stored at index 2 * 4 + 1 = 9.*



To reach the cell at column x and line y (both starting at 0), skip y full lines, then move x characters to the right:

```
grid[y * width + x]
```

IV.4 Expected output

Your program takes exactly two arguments: the width and the height of the grid, both made only of digits.

- The grid must always be large enough to keep at least one `FILLER` around the word. With `len` the length of `BAZINGA`:
  - its width is the largest value between `width` and `len + 2`;
  - its height is the largest value between `height` and `3`.
- Allocate the grid with one single `malloc`, and fill it with `FILLER`.
- Write `BAZINGA` in the grid, using its final size and integer divisions (lines and columns are numbered from 0):
  - on line `height / 2`,
  - starting at column `(width - len) / 2`.
- Display the grid line by line, each line followed by a newline, then free it and return 0.

The arguments are checked from left to right: only the **first** error found is displayed. Every error message is displayed on the **error output**, followed by a newline, and your program returns 1:

Situation	**Message**
wrong number of arguments	`Error: usage ./a.out <width> <height>.`
invalid width	`Error: width must contain only digits.`
invalid height	`Error: height must contain only digits.`
`malloc` fails	`Error: malloc failed.`



`You don't have to handle huge values: during the evaluation, width and height will never exceed 10000.`

IV.5 Examples

```
$> ./a.out 15 5 | cat -e
...............$
...............$
....BAZINGA....$
...............$
...............$
$> ./a.out 12 4 | cat -e
............$
............$
..BAZINGA...$
............$
$> ./a.out 0 0 | cat -e
.........$
.BAZINGA.$
.........$
$>
```

Errors are displayed on the error output, so they are not affected by `| cat -e`:

```
$> ./a.out 15
Error: usage ./a.out <width> <height>.
$> ./a.out 15 abc
Error: height must contain only digits.
$> ./a.out -3 5
Error: width must contain only digits.
$> echo $?
1
$>
```

Chapter V

Submission and peer-evaluation

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the peer evaluation. Make sure to double-check the names of your files and folders to ensure they are correct.



`Your files must be placed in a folder named ex03 at the root of your repository: ex03/bazinga.c and ex03/bazinga.h`

During the evaluation, a brief **modification of the exercise** may be requested. This could involve a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature.

This step is meant to verify your actual understanding of the exercise. It should be feasible within a few minutes, in any development environment you choose.

For this exercise, your evaluator will for example change the values of the macros. With `BAZINGA` set to `"42"` and `FILLER` set to `'#'`, your program must display:

```
$> ./a.out 6 3 | cat -e
######$
##42##$
######$
$>
```